

DigitalSreeni Image Annotator — User Manual

Version 0.9.0

A desktop tool for annotating images and videos with polygons, rectangles, keypoints, and AI-assisted masks — with built-in support for fine-tuning your own SAM 2 model and training YOLO models on your annotations.

A table of contents is generated automatically when this file is viewed on GitHub (the outline icon) or converted to HTML/PDF (see the accompanying `USER_MANUAL.html`).

Introduction

DigitalSreeni Image Annotator is a PyQt6 desktop application for creating image and video annotations for computer-vision datasets — object detection, instance segmentation, semantic segmentation, and keypoint/pose estimation.

It supports three complementary ways of creating annotations:

- **Manual** — polygon, rectangle, paint-brush, and eraser tools.
- **AI-assisted, prompted by you** — SAM 2 (click points or draw a box, get a mask), Grounding-DINO and SAM 3 (describe the object in plain English, get boxes/masks).
- **AI-assisted, from a model you trained** — fine-tune SAM 2 or train a YOLO model on your own annotations, then use it to pre-label the rest of your dataset.

It also natively handles the kinds of images a manual annotation tool usually struggles with: multi-page TIFF stacks, Zeiss CZI microscopy files, and video files (MP4/AVI/MOV) — each treated as a navigable stack of 2D slices/frames.

Everything lives inside a **project** (an `.iap`/JSON file) that stores your images, classes, and annotations together, with autosave and a full undo/redo history for every edit.

Installation

```
pip install digitalsreeni-image-annotator
```

This pulls in PyTorch, Ultralytics (SAM 2 / YOLO), Transformers (Grounding-DINO), OpenCV, Shapely, and MLflow — there is nothing extra to install to get AI-assisted annotation working.

Run the app with any of:

```
digitalsreeni-image-annotator  
sreeni  
python -m digitalsreeni_image_annotator.main
```

GPU acceleration (NVIDIA)

The default PyPI install on Windows ships a **CPU-only** PyTorch wheel. If you have an NVIDIA GPU, reinstall PyTorch from the CUDA index for a large speed-up in SAM, Grounding-DINO, SAM 3, and training:

```
pip uninstall -y torch torchvision  
pip install torch torchvision --index-url https://download
```

If `cu128` reports "no matching distribution," try `cu124`. Verify with:

```
python -c "import torch; print(torch.cuda.is_available()),
```

Older NVIDIA GPUs (GTX 10-series / Pascal, or Maxwell): PyTorch $\geq$ 2.8 wheels drop kernels for compute capability < 7.0 . The app detects this automatically, warns you once, and falls back to CPU instead of crashing. To keep using such a GPU, install an older PyTorch build instead:

```
pip install torch==2.4.1 torchvision==0.19.1 --index-url
```

Linux runtime libraries

Qt 6 requires: `sudo apt install libxcb-cursor0 libegl1 libgl1 libxcb-xinerama0 libxkbcommon-x11-0` (Debian/Ubuntu).

The Interface at a Glance

Area	What's there
Menu bar	Project, Settings, Tools, Help
Left sidebar	Tool buttons (Polygon / Rectangle / Paint / Eraser / SAM-box / SAM-points / Keypoint), class list, SAM model picker, Grounding-DINO / SAM 3 panel
Center	The canvas — the current image or video frame, with zoom/pan, and (for videos) a scrub timeline below it
Right side	Image list (with an annotation-status filter and A–Z sort) and, below it, the slice list for stacks/videos
Bottom-left	Annotations table for the current image: ID, Class, Area, Detail %

The canvas has two implicit modes:

- **A tool is active** (a tool button is checked) — clicks draw with that tool.
- **No tool is active — Selection mode** — clicking selects an existing annotation; this is where you delete, resize, move, and vertex-edit shapes.

Pressing **Esc** always returns you to selection mode, cancelling anything in progress first.

Starting a Project

1. **Project** → **New Project** (Ctrl+N).
2. **Add Images/Videos** to bring in images, TIFF stacks, CZI files, or video files.
3. **Add Classes** to define the categories you'll annotate.
4. Pick a class, pick a tool, start annotating.
5. **Project** → **Save Project** (Ctrl+S) — or just let autosave handle it.

Opening an existing project (Ctrl+O) that references images which have moved will prompt you to relocate them; located files are copied back into the project. If you skip relocating a missing image, its annotations are dropped.

Loading Images and Videos

Add Images / Videos opens a file picker accepting `.png .jpg .bmp .tif .tiff .czi .mp4 .avi .mov` in one dialog.

- **Regular images** (PNG/JPG/BMP) load directly.
- **TIFF stacks and CZI files** trigger the dimension-assignment dialog — see [§15](#).
- **Videos** (MP4/AVI/MOV) load as a stack whose "slices" are individual frames, decoded on demand (nothing is bulk-extracted to disk) — see [§13](#). The status bar reports total frame count,

resolution, and fps once loaded.

The image list on the right can be **filtered** (with/without annotations) and stays **alphabetically sorted**; filtering only hides rows, it never removes them, so keyboard navigation and your place in the list stay stable.

Removing an entry from the list shows "Remove Video" instead of "Remove Image" for video entries. Loading a compressed TIFF that needs the optional `imagecodecs` codec package shows an actionable "pip install imagecodecs" message instead of crashing, if that package isn't present (it is installed by default with the app).

Classes

- **Add Classes** to create one or more categories, each with an auto-assigned color (colors are chosen so red is not the first class — it's reserved further down the palette for visibility reasons).
 - Right-click a class in the class list for **Rename**, **Change Color**, **Delete**, and — for pose annotation — **Define Keypoint Schema** (see [§14](#)).
 - The checkbox next to each class **toggles its visibility** on the canvas without deleting anything.
 - **Change the class of an annotation**: select it (canvas or table) and use the Change Class control — handy for correcting SAM/ DINO misclassifications quickly. This is blocked for keypoint/pose instances when the schemas differ.
-

Manual Annotation Tools

Tool	How to use it
Polygon	Click to place vertices around the object; press Enter to close and commit, Esc to cancel.

Rectangle	Click-drag a bounding box; release to commit.
Paint Brush	Paint a freeform mask; adjust brush size with - / =. Small stray specks (<10 px) are filtered out automatically. Converts to a polygon on commit.
Eraser	Subtracts from an existing mask (e.g. to trim SAM over-segmentation). Same size controls as the brush.

All drawn shapes are automatically clipped to the image boundary — you cannot commit an annotation that spills outside the frame.

Merge: select two or more annotations (canvas or table, see §8) and click Merge — their union becomes one annotation, replacing the originals. There's no confirmation dialog; if it's wrong, **Ctrl+Z** undoes it.

Selecting, Editing, and Deleting Annotations

When no tool is active, the canvas behaves like a normal selection pointer, and it is **unified with the annotations table** — selecting on one side selects on the other.

Action	Result
Click a mask	Select it (replaces any prior selection)
Shift+Click	Toggle that mask in/out of the selection
Drag on empty canvas	Rubber-band select everything the box touches
Shift+Drag	Add the rubber-band catch to the current selection
Click empty space	Clear the selection
Delete	Remove the selected annotation(s) — instant, no confirmation, undoable

Resizing and moving — when exactly **one** shape is selected, eight square handles appear (4 corners + 4 edge midpoints):

- Drag a **handle** → resize (a box edits its `[x, y, w, h]`; a polygon scales its vertices around the opposite anchor).
- Drag **inside** the shape → move it.
- Releasing clamps the result back inside the image if needed.

Vertex editing — double-click a polygon to enter vertex-edit mode and reshape it point-by-point; press Enter to commit (clamped to the image) or Esc to revert.

Ctrl is reserved for panning the canvas — multi-select always uses Shift.

Undo / Redo

Ctrl+Z undoes the last annotation edit; **Ctrl+Y** (or **Ctrl+Shift+Z**) redoes it. This covers drawing, deleting, merging, resizing/moving, vertex edits, class changes, and every AI-assisted accept (SAM, DINO, SAM 3, YOLO prediction, tracking).

Undo/redo is per-image/per-frame — each frame of a video, or each slice of a stack, keeps its own history. Because undo is always available, destructive actions (Delete, Merge) no longer show "Are you sure?" or "N deleted" dialogs — if something goes wrong, Ctrl+Z is the safety net.

SAM 2 — AI-Assisted Segmentation

1. Pick a model from the **SAM model dropdown**: SAM 2 or SAM 2.1, each in tiny/small/base/large. **Tiny or small are recommended** — large can crash the app on machines with limited RAM/VRAM.
2. First use of any given model downloads its weights (cached

afterward in the working directory) — a few seconds to a minute depending on your connection.

3. Choose a prompt style:
 - **SAM-box**: draw a rectangle around the object.
 - **SAM-points**: left-click positive points *inside* the object; right-click negative points to exclude regions.
4. SAM 2 shows its top-scoring mask as an orange preview. **Enter** accepts it into your annotations; **Esc** discards it and lets you try again (redraw the box, or add/adjust points).
5. If SAM 2 catches only part of an object, finish the rest with the Polygon or Paint tool and **Merge** the two into one annotation. If it over-segments, clean the edges with the **Eraser**.

If your GPU's compute capability is too old for the installed PyTorch build, the app detects it, shows a one-time warning, and runs inference on CPU automatically rather than crashing.

Tools → Unload AI Models (Free GPU Memory) drops the cached SAM/DINO/SAM 3 models from memory and resets both model selectors — useful before switching tasks on a memory-constrained GPU. A restart is still needed to fully reclaim the last few hundred MB PyTorch's CUDA context holds onto.

Grounding-DINO — Text-Prompted Detection

Grounding-DINO finds objects from a **text description** instead of clicks; the resulting boxes are then automatically refined into masks by SAM 2.

Setting it up

1. In the DINO panel, pick a model: **grounding-dino-base**, **grounding-dino-tiny**, or **Custom / fine-tuned (browse)** for your own checkpoint. Preset models download from Hugging Face on first use if not already cached.
2. For each class you want detected, add one or more **phrases** in the phrase editor (e.g. class "drone" → phrases "drone",

"quadcopter", "octocopter"). The class name itself is always included and can't be removed.

3. Tune per-class thresholds in the table: **Box thr**, **Txt thr**, **NMS thr** (higher NMS = more aggressive duplicate removal).

Detecting

- **Detect Current Image** runs detection + SAM refinement on the frame you're viewing. Results appear as an orange overlay.
- **Detect All Images** runs the same pipeline across every image *and every slice of every loaded stack/video*.
- The **auto-accept dropdown** ("Review before accepting" vs. "Auto-accept all detections") governs both buttons identically.

Reviewing

- **Enter** accepts the visible detections into your annotations (new classes are created automatically if needed).
- **Esc** discards them.
- These keys work even if focus is on a list or button elsewhere in the window — you don't need to click the canvas first.
- In batch mode with "Review before accepting" selected, you're walked through each image/frame's detections one at a time.

Merge COCO for Training (Tools menu) merges the DINO+SAM annotations you've accumulated across images into a single training-ready COCO JSON — handy for bootstrapping a fine-tuning or YOLO-training dataset from AI-assisted labels.

SAM 3 — Text-Prompted Segmentation

SAM 3 sits in the **same dropdown as Grounding-DINO**, as the entry **"SAM 3 (text prompt)"** — it's a drop-in alternative producer for the identical review workflow (same phrase panel, same Enter/Escape accept/reject, same batch mode), except it goes straight from text to

mask without a separate SAM 2 refinement pass.

One-time setup

SAM 3's weights (`sam3.pt`, ~3.45 GB) are **gated on Hugging Face** and are **never auto-downloaded**. To use it:

1. Request access to SAM 3 on Hugging Face.
2. Place the downloaded `sam3.pt` file in the app's working directory or its `models/sam/` folder.
3. Select **"SAM 3 (text prompt)"** in the model dropdown. If the file isn't found yet, the status line reads *"SAM 3 weights (sam3.pt) not found. Request access on Hugging Face, then place sam3.pt in the working directory or the models/sam/ folder."* and detection stays disabled until it is.
4. Once found, selecting the entry loads the model (status shows *"Loading SAM 3 ..."* then *"Ready: SAM 3 (text prompt)"*).

The very first SAM 3 detection also needs network access once: Ultralytics auto-installs two extra packages (`clip`, `timm`) the model needs, then continues in the same run without a restart.

Everything downstream — the phrase list, thresholds, Detect Current Image / Detect All Images, review, batch mode — works exactly as described for Grounding-DINO in §11.

Video Annotation and Object Tracking

Loading and navigating a video

Add an `.mp4`, `.avi`, or `.mov` file the same way as an image (§5). It's treated as a stack whose frames are its slices, decoded one at a time — nothing is pre-extracted to disk, so even long videos load quickly.

- A **scrub timeline** appears below the canvas whenever a video is

the active item. It shows a `F <i>/<N>` • `MM:SS / MM:SS` position readout and colors a thin marker strip under frames that are **annotated**, **tracked**, or **need review** (needs-review takes visual priority over tracked, which takes priority over plain annotated).

- Drag the timeline, or use **Up/Down** on the slice list, to move one frame at a time; **Home/End** (with the slice list or canvas focused) jump straight to the first/last frame.
- There is no play/autoplay — annotation is frame-by-frame, by design.

Annotating

Every manual tool and every AI-assisted tool (SAM 2, Grounding-DINO, SAM 3) works on a video frame exactly as it does on a still image.

Annotate a frame, move to the next, repeat — or use tracking to do that for you (below).

Tracking a segmented object across frames

Once you've drawn or accepted a mask on one frame, you can propagate it forward/backward through the rest of the video using SAM 3's video predictor:

1. Load a video and load the **SAM 3** model (see §12) — tracking needs SAM 3, not SAM 2.
2. **Select exactly one mask annotation** on the current frame (a keypoint/pose instance can't be tracked — it has no mask to propagate).
3. **Tools** → **Track Selected Object...** (enabled only when the three conditions above are met). A dialog asks for a **confidence threshold** (default 0.5).
4. A progress dialog ("Tracking object across frames...", indeterminate — no percentage/ETA) runs while SAM 3 propagates the mask **both forward and backward** from the seed frame through the whole clip; **Cancel** stops it early.
5. Results are routed automatically:
 - **High-confidence frames** (score $\geq$ threshold) are written

straight into your project as regular annotations (tagged internally so they can be bulk-undone — see below). In practice SAM 3's video predictor reports full confidence on nearly every frame where it finds the object at all (absence shows up as an empty mask, not a low score), so most tracked frames land here.

- **Low-confidence frames** ($0 < \text{score} < \text{threshold}$) are queued for review through the same Enter/Escape overlay used for DINO batch review. If any exist, you're asked *"Tracking finished. N frame(s) had low-confidence results. Review them now?"*.
 - Frames where no object was found produce nothing.
6. The timeline's marker strip updates to show which frames were tracked.

Very long videos are decoded entirely into memory for the duration of a tracking run (bounded by clip length) — fine for typical annotation clips, but worth keeping in mind for a very long source video.

Undo: every tracked frame is individually undoable with Ctrl+Z on that frame. For undoing the whole run at once, use **Tools** → **Undo Last Track**, which removes every annotation from the most recent tracking run in one action (pre-existing annotations on those frames are left alone).

Exporting

Tools → **Export Annotated Video Frames...** saves every frame that has at least one annotation as a PNG (`<video>_F00003.png`, etc.) into a folder you choose — a quick way to pull a labeled image set out of an annotated video for use elsewhere (e.g. as input to the Tools listed in §23, or a COCO/YOLO export via the normal Export dialog, which treats video frames like any other slice).

Keypoint / Pose Annotation

Keypoint annotation lets you mark a **named skeleton** on instances of a class (people, animals, tools — anything with a consistent point layout), following the COCO pose model.

1. Define a schema (once per class)

Right-click the class → **Define Keypoint Schema**:

- **Keypoints (ordered) table** — add each point's **Name** and, optionally, its horizontal-**flip partner** (used for training-time flip augmentation; leave blank for points on the symmetry axis, e.g. "nose").
- **Skeleton** — define the edges connecting points (drawn as lines between them).

Once any instance of a class exists, its point *count* is locked, but names/skeleton/flip-partners stay editable.

2. Place an instance

Activate the **Keypoint** tool (blocked with a warning if the current class has no schema yet). Place the class's K points **in schema order**:

Input	Result
Left-click	Place the next point, visible
Right-click (or Shift+left-click)	Place the next point, occluded
Backspace	Remove the last placed point
Enter (before all K are placed)	Finish early — remaining points are marked "not placed"

The instance auto-finishes once all K points are placed. It renders as the skeleton lines plus markers colored by visibility, with a faint bounding box and label.

3. Editing

With the instance selected:

- **Drag a point marker** → move that single point.
- **Right-click a point marker** → toggle visible ↔ occluded.
- **Drag a box handle, or drag inside the instance box** → transform the *whole pose* (move/resize all points together), the same handle mechanism used for regular shapes.

Merge and cross-schema class-change are both blocked for pose instances (merging keypoints has no sensible meaning).

4. Export / import

- **COCO JSON** — each pose class's category gains `keypoints` (names), `skeleton`, and `flip_idx`; each instance exports as a COCO keypoint annotation (no `segmentation`).
- **YOLO-pose** — label lines gain the extra `(x, y, v)` triplets and `data.yaml` gains `kpt_shape/flip_idx`. Because YOLO-pose has **one schema for the whole dataset**, exporting a mix of different keypoint counts, or mixing a pose class with a non-pose class, is rejected up front with a clear error — nothing partial is written to disk.
- Both formats import back with their schema recovered automatically.

5. Training and predicting pose models

Loading a `*-pose.pt` checkpoint in the YOLO Training menu and training proceeds exactly like detection/segmentation (§21) — the app checks the model/dataset agree on being pose-shaped *before* training starts. Predictions come back as full keypoint instances (visibility is always reported as "visible" — Ultralytics doesn't return true 3-state occlusion) and go through the same accept/reject review as any other prediction.

Multi-Dimensional Images (TIFF Stacks & CZI)

Adding a TIFF or CZI file with more than two dimensions opens the **dimension-assignment dialog**. For a well-formed ImageJ-style TIFF, the axis order is read from the file's metadata and the dialog is pre-filled — usually you can just click OK.

Assign each axis a meaning: **T** (time), **Z** (depth), **C** (channel), **S** (scene), **H/W** (the 2D image plane). The app then extracts every combination as an individually-navigable 2D slice, named like:

```
stack.tif_T0_Z5_C0
```

- **Up/Down** arrows step through slices.
 - Each slice keeps its **own independent annotations**.
 - The slice list on the right shows every extracted slice for the currently selected stack.
-

Mask Complexity — Detail %

SAM/DINO masks are often dense (hundreds of vertices). In the Annotations table, each row has a **Detail %** spinbox (1–100, default 100 = raw):

- Lowering it thins the polygon (Douglas-Peucker simplification) — useful for smaller, cleaner label files.
 - It's fully **reversible**: the first time you simplify, the original dense polygon is preserved internally; setting it back to 100 restores the exact original.
 - Exports always use the *currently displayed* (possibly simplified) version.
-
-

Saving, Loading, and Managing Projects

- **Ctrl+S** saves; **Save Project As...** saves a copy under a new name; **Ctrl+O** opens.
- **Autosave** runs continuously as you work (it is automatically suppressed while a project is loading, so an autosave can never race a load and corrupt the file).
- **Project Details** (Ctrl+I): view creation/modified dates and per-image info, and write free-form project notes (saved automatically as you type).
- **Search Projects** (Ctrl+F): search across every project you've worked on, by project name, class names, image names, and notes, using **AND/OR/parentheses**, e.g.:
 - cells AND dog
 - cells OR bacteria
 - cells AND (dog OR monkey)
 - (project1 OR project2) AND (cells OR bacteria)

Double-click a result to open that project.

Project files store **absolute paths** to your images — projects aren't portable between machines/drives without relocating images on open.

Importing Annotations

Import Annotations with Images loads an existing **COCO JSON** file (images must sit alongside it) or a **YOLO v5+/v8/v11**-structured dataset directory into your current project, including any recoverable keypoint schema.

Exporting Annotations

Export Annotations offers:

Format	Notes
COCO JSON	Copies images into an <code>images/</code> folder alongside the JSON. Keypoint-schema-aware.
YOLO v8/v11 (v5+ structure)	<code>data.yaml + images/{train, val} + labels/{train, val}</code> . You're prompted for a validation split % (0 = everything into train); the split is a deterministic hash of filenames so a requested split is never accidentally empty. Pose-aware (adds <code>kpt_shape/flip_idx</code> when applicable; rejects inconsistent pose exports before writing anything).
YOLO v4 (legacy)	Older flat YOLO <code>.txt</code> format.
Labeled images	Colored overlay visualizations (PNG).
Semantic labels	Single-channel PNGs where pixel value = class ID.
Pascal VOC	XML, bounding boxes.

Exports look up each annotation's source image by exact filename first, falling back to substring matching only for older projects — so a class named `bee` never accidentally grabs the file for `honeybee`.

SAM 2 Fine-Tuning (Custom Model Training)

If the built-in SAM 2 weights aren't segmenting your specific kind of imagery well, you can fine-tune SAM 2 on your own annotations instead of relying on the generic pre-trained model. This is separate from — and

complementary to — YOLO training (§21).

Starting a run

SAM Fine-Tune (beta) → Train on Current Project... (or **Prepare SAM Dataset... / Train from Dataset Folder...** for an offline, inspectable dataset export-then-train flow — `export_sam_dataset` writes `images/` + a `manifest.json` you can review before training).

A configuration dialog collects:

- **Base model** to fine-tune from.
- **Epochs, peak learning rate, batch size.**
- **Prompt type**: bounding box or point.
- **Train/validation split %** and **early-stop patience.**
- **Warmup** → **cosine LR schedule** toggle.
- Whether to **also fine-tune the image encoder** (default is decoder-only — faster, lower VRAM, and usually enough; unfreezing the encoder too is for heavily domain-shifted imagery with more data).

If no GPU is usable, you're warned up front and can back out rather than starting a very slow CPU run.

While it trains

- SAM tools are locked during training (the trainer needs its own model instance on the GPU).
- A progress dialog reports **train loss, validation loss**, and **learning rate** per epoch, with a **Stop** button.
- Early stopping kicks in based on the patience you configured; the **best validation checkpoint** is what gets saved, not necessarily the last epoch.
- Every run is automatically logged to **MLflow** (§22) — no checkbox needed, tracking is always on.

Using the result

Once training finishes, the fine-tuned model is registered and appears in the **SAM model dropdown** as "**★ <your model name>**", ready to use with SAM-box/SAM-points exactly like a built-in model.

YOLO Training and Prediction

Train a YOLO model directly from your current annotations, covering **detection, segmentation, and pose** (previously segmentation-only).

1. **YOLO (beta) → Training → Train Model.** The dialog collects epochs, image size, a warmup→cosine LR toggle, peak learning rate, and early-stop patience.
2. Training runs on a background thread; a progress dialog shows per-epoch **train loss**, and after each validation pass, **val loss + mAP50 + mAP50-95** (plus pose-specific `val/pose_loss / val/kobj_loss` when training a pose model).
3. Loading a pose checkpoint against a non-pose dataset (or vice versa) is caught and reported clearly **before** training starts, not partway through.
4. On completion, the run is logged to MLflow automatically (§22), the result is pruned to just `best.pt + data.yaml` (the rest of the diagnostics live in the MLflow run instead of duplicated on disk), and it's registered for prediction as "**★ <project name>**".
5. **Prediction Settings → Load Model** to pick a trained (or external) model + its `data.yaml`, then **Predict with YOLO Model** to run it on the current image. Detections/masks/poses appear as reviewable temp annotations (same Enter/Escape-style accept flow), and are added under a "`Temp-<class>`" label until accepted.

Loading a model trained on classes that don't match the loaded `data.yaml` shows a clear mismatch message instead of crashing.

Known limitation: YOLO training is not currently supported for multi-dimensional images (TIFF stacks / CZI slices) — single images only.

Experiment Tracking with MLflow

Every SAM fine-tuning run and every YOLO training run is tracked in **MLflow** automatically — there's no "enable tracking" checkbox; a broken MLflow install degrades gracefully to "this run wasn't tracked" rather than blocking training.

- **Settings** → **Experiment Tracking** → **MLflow Settings...** lets you point tracking at a different store location (default: an `mlruns` folder inside your open project, or the current working directory if no project is open).
 - **Settings** → **Experiment Tracking** → **Open MLflow UI** launches MLflow's web dashboard in your browser so you can compare runs, loss curves, and hyperparameters across attempts.
 - When a training run starts, a clickable link to that specific run's page appears in the training progress dialog.
-

Tools Menu — Dataset & Image Utilities

Each of these opens its own guided dialog:

Tool	Purpose
Annotation Statistics	Per-class counts and area statistics for the current project, with charts.
COCO JSON Combiner	Merge multiple COCO JSON annotation files into one.
Dataset Splitter	Split a dataset into train/val/test sets with configurable ratios.
Merge COCO for Training	Merge accumulated Grounding-DINO/SAM annotations across images into one training-ready COCO JSON.
Stack to Slices	Convert a multi-dimensional stack into individual 2D image files on disk.

Image Patcher	Split large images into smaller overlapping/non-overlapping patches.
Image Augmenter	Apply rotation/flip/brightness/etc. transformations to expand a dataset, with a live preview. Augmented polygons that partially leave the frame are geometrically clipped to the boundary (and dropped if they end up entirely outside it).
Slice Registration	Align slices in a stack using multiple registration algorithms and reference-frame options.
Stack Interpolator	Adjust Z-spacing in an image stack — e.g. generate an isotropic volume from anisotropic microscopy data, with memory-efficient processing for large stacks.
DICOM Converter	Convert DICOM files to TIFF (single stack or individual slices), preserving metadata; can also export metadata to JSON.
Export Annotated Video Frames...	See §13.
Track Selected Object... / Undo Last Track	See §13.
Unload AI Models (Free GPU Memory)	See §10.

Appearance: Dark Mode and Font Size

- **Settings** → **Toggle Dark Mode** (Ctrl+D) switches the whole UI, including annotation rendering colors chosen for visibility against the dark background.
- **Settings** → **Font Size** offers fixed presets (Small...XXL), or fine

control:

- **Ctrl+Shift+=** (or **Ctrl++**) — increase font size
 - **Ctrl+Shift+-** (or **Ctrl+-**) — decrease
 - **Ctrl+Shift+0** — reset to default
 - Font size (8–24pt) and dark mode are saved as personal preferences (not part of the project file), so they persist across projects and sessions on the same machine.
 - Canvas overlays (selection handles, point markers, label text) scale with the font size setting independently of image zoom, so they stay a consistent on-screen size as you zoom in/out of the image.
-

Keyboard Shortcuts Reference

Global

Shortcut	Action
Ctrl+N	New Project
Ctrl+O	Open Project
Ctrl+S	Save Project
Ctrl+Shift+S	Save Project As...
Ctrl+W	Close Project
Ctrl+I	Project Details
Ctrl+F	Search Projects
Ctrl+Alt+S	Annotation Statistics
Ctrl+D	Toggle Dark Mode
Ctrl+Z	Undo
Ctrl+Y / Ctrl+Shift+Z	Redo
Ctrl+Shift+= / Ctrl++	Increase UI font size

Ctrl+Shift+- / Ctrl+-	Decrease UI font size
Ctrl+Shift+0	Reset UI font size
F1	Help window

Canvas

Shortcut	Action
Ctrl+Wheel	Zoom in/out
Ctrl+Drag	Pan
Click (no tool)	Select the mask under the cursor
Shift+Click (no tool)	Toggle that mask in the selection
Drag (no tool)	Rubber-band select; Shift+Drag adds
Drag a handle (one shape selected)	Resize
Drag inside (one shape selected)	Move
Double-click	Enter vertex-edit mode
Delete	Delete selected annotation(s)
Enter	Finish/accept current shape or AI prediction
Esc	Cancel current action and return to selection mode
Up / Down	Navigate slices / video frames
Home / End	Jump to first / last frame (video, slice list or canvas focused)
- / =	Adjust paint brush / eraser size

Keypoint tool

Input	Action
Left-click	Place next point, visible
Right-click / Shift+left-click	Place next point, occluded
Backspace	Remove last placed point
Enter	Finish pose early (unplaced points marked "not placed")
Right-click a placed point (idle)	Toggle visible ↔ occluded

Troubleshooting & Known Limitations

- **SAM 2 large crashes / hangs** on machines with limited RAM — use SAM 2/2.1 tiny or small instead.
 - **YOLO training doesn't support multi-dimensional images** — export/flatten to single images first (Stack to Slices, §23) if you need to train on stack data.
 - **SAM 3 requires manually placed weights** — it will never auto-download `sam3.pt`; see §12.
 - **Out-of-memory loading a SAM model** now shows an actionable "pick a smaller model" message instead of a raw crash.
 - **Projects aren't portable across machines** — they store absolute image paths; use the missing-image relocation prompt on open if you've moved files.
 - **GPU not being used despite being detected**: on very old NVIDIA GPUs, the installed PyTorch build may lack kernels for that GPU's compute capability — the app falls back to CPU automatically and warns once; see the GPU section in §2 for the older-PyTorch workaround.
 - If the app seems to be running out of GPU memory across a long session with both SAM and DINO/SAM 3 loaded, use **Tools** → **Unload AI Models** — and remember a full reclaim needs an app restart, not just the unload.
-

Glossary

- **Annotation** — one marked region (polygon, box, or keypoint instance) tied to a class.
- **Class** — a category label with its own color; can carry a keypoint schema to become a "pose class."
- **SAM 2 / SAM 3** — Meta's Segment Anything models; SAM 2 is prompted with points/boxes, SAM 3 with text.
- **Grounding-DINO** — a text-prompted object *detector* (boxes only); its boxes are refined into masks by SAM 2 in this app.
- **Detail %** — reversible polygon simplification control per annotation (§16).
- **Keypoint schema** — the named point layout + skeleton that makes a class a "pose class" (§14).
- **Fine-tuning** — continuing to train SAM 2 on your own annotations rather than relying on the generic pre-trained weights (§20).
- **MLflow run/experiment** — a tracked training attempt with its parameters, metrics, and artifacts (§22).
- **Slice** — one 2D plane extracted from a multi-dimensional TIFF/ CZI stack or one frame of a video.
- **Track run** — one invocation of "Track Selected Object," which can be undone as a unit (§13).

This manual covers DigitalSreeni Image Annotator v0.9.0. For the latest source, issues, and releases, see the project's [GitHub repository](#).